# Creating Linux Virtual Servers

*Wensong Zhang, Shiyao Jin, Quanyuan Wu*
National Laboratory for Parallel & Distributed Processing
Changsha, Hunan 410073, China
wensong@iinchina.net
http://proxy.iinchina.net/~wensong

## 개 요

빠른 넷트웍에 연결된 서버(server)들의 클러스터(cluster)는 성능이 뛰어나고 (high performance) 가용성이 높은 (highly available) 서버를 구축하는데 유효한 구조임이 판명되고 있다. 이러한 형태의 느슨하게 연관된 구조는 단일 프로세서(processor) 시스템이나 엄격한 다중 프로세서 시스템보다 확장성이 뛰어나고 (more scalable) 비용-효율성이 좋으며 (more cost-effective) 신뢰성이 높다 (more reliable).

이 글은 어떻게 리눅스 가상 서버들을 만드는지 설명하고 있다. 가상 서버는 실제 서버들의 클러스터에 구축된 성능이 뛰어나고 가용성이 높은 서버이다. 두가지 방법의 IP-계층 부하 분산 방식(load balancing)이 클러스터의 병렬 서비스를 단일 IP 주소상에 하나의 가상 서비스로 보이도록 개발되었다. 하나는 넷트웍 주소 번역(Network Address Translation; NAT)에 의한 가상 서버이고, 다른 하나는 IP 터널링(IP tunneling)에 의한 가상 서버이다. 현재 네가지 작업 할당 방식들(scheduling algorithms)이 각기 다른 응용 상황에 맞춰 개발되어 있다. 확장성은 클러스터에 노드(node)를 투명하게 추가하거나 제거하는 방법으로 이루어진다. 높은 가용성은 노드 또는 데몬(deamon)의 장애(failure)나 시스템의 구조 변화를 적절히 감지해냄으로써 이루어진다.

번역 : 리눅스코리아(주) 개발팀 이선중

# 1. 소개

인터넷의 폭발적 성장과 인터넷이 우리 생활에서 차지하는 중요도가 커짐에 따라 인터넷의 정보 전송량도 극적으로 증가하고 있어 매년 100%를 넘는 증가가 이루어진다. 유명한 인터넷 사이트의 부하(load)가 급격히 증가하고 있고 이미 하루에 억대의 방문(hit)을 받는 곳도 있다. 점점 더 많은 관리자들이 그들 서버의 성능상 병목 문제와 직면하고 있지만 서버의 액세스(access) 용량을 늘려도 얼마 지나지 않아 쉽게 과부하가 되어버린다. 요즘은 점점 더 많은 회사들이 그들의 사업을 인터넷상으로 옮기고 있어 서버가 제공하는 서비스의 어떠한 단절/중단도 사업상 손실을 의미하게 되었다. 이러한 서버들의 높은 가용성 (availability)은 점점 더 중요해지고 있다. 그러므로 확장성이 있고(scalable) 높은 가용성을 가진 서버에 대한 요구가 절실히 증가하고 있다. 이러한 형태의 서버들의 필요 조건을 요약하자면 다음과 같다:

* 추가 확장성
* 높은 가용성
* 비용-효율성

서버를 더 높은 성능의 서버로 개선(upgrade)하는 단일 서버 솔루션(solution)은 이 필요 조건을 맞추기에 한계가 있다. 개선 작업이 복잡하고 원래의 기계가 낭비될 수도 있다. 만일 요구(request)가 증가된다면 오래 지나지 않아 과부하가 되어 더 높은 성능의 서버로 다시 개선해주어야한다. 이 서버에 장애가 일어나면 전체 시스템에 영향을 미치는 곳(single point of failure)이다. 개선해야할 목표가 높아지면 높아질수록 비용이 더욱 더 많이 든다.

빠른 넷트웍에 연결된 서버들의 클러스터는 성능이 뛰어나고 가용성이 높은 서버를 구축하는데 유효한 구조임이 판명되고 있다. 이러한 형태의 느슨하게 연관된 구조는 단일 프로세서 시스템이나 엄격한 다중 프로세서 시스템보다 확장성이 뛰어나고 비용-효율성이 좋으며 신뢰성이 높다. 그러나 클러스터내 병렬 서비스의 투명성(transparency), 확장성, 그리고 높은 가용성을 제공해야하는 어려움이 있다.

이 글은 어떻게 리눅스 가상 서버들을 만드는지 보여준다. 가상 서버는 느슨하게 연관된 독립된 서버들의 클러스터에 구축된 확장 가능하고 가용성이 높은 서버이다. 클러스터 구조는 클러스터 밖의 클라이언트(client)에 투명하다. 클라이언트 응용 프로그램은 마치 클러스터가 하나의 고성능, 고가용성의 서버인것처럼 클러스터와 상호작용한다. 클라이언트는 클러스터와의 상호작용에 의해 침범 받지 않으며 변형될 필요가 없다. 일반적인 가상 서버의 구조는 그림 1과 같다.

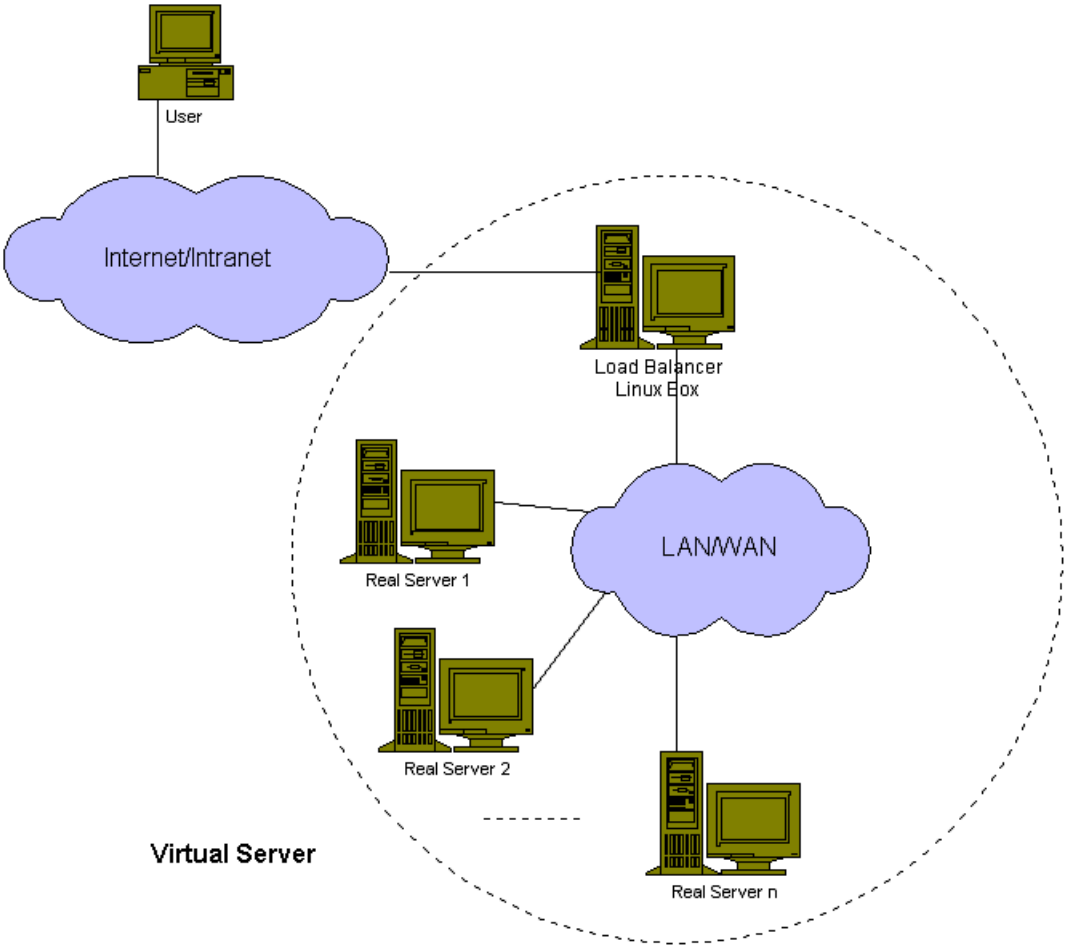



그림 1. 가상 서버의 일반적인 구조

실제 서버들은 고속 LAN 또는 지역적으로 분산된 WAN으로 연결된다. 실제 서버들의 외부 연결부는 부하 분산기(load balancer)이다. 부하 분산기(load balancer)는 요구(request)를 각기 다른 서버들에게 작업 할당(schedule)하고 클러스터의 병렬 서비스들을 하나의 IP 주소상의 하나의 가상 서비스로 보이도록 만든다. 높은 가용성은 노드(node) 또는 데몬(daemon)의 장애(failure)나 시스템의 구조 변화(reconfiguring)를 적절히 감지해냄으로써 이루어진다.

이 글의 나머지 부분은 다음과 같이 구성되어있다: 2장에서 연관된 작업들을 다룬다. 3장에서 넷트웍 주소 번역(Network Address Translation)에 의한 가상 서버와 IP 터널링(IP tunneling)에 의한 가상 서버의 구조와 작동 원리, 또한 이들의 장단점을 설명한다. 4장에서 리눅스 가상 서버에서 개발된 네가지 작업 할당 방식들(scheduling algorithms)을 설명한다. 5장에서 리눅스 가상 서버의 고 가용성 문제를 다룬다. 마지막으로 결론과 미래에 해야할 일이 6장에 있다.

# 2. 연관된 작업들

클라이언트/서버 환경(client/server applications)에서 한쪽 끝은 클라이언트고 다른 한쪽 끝은 서버가 된다. 그리고 중간에 웹 서비스에서의 프락시 서버같은 프락시(대리인 代理人; proxy)가 있을 수 있다. 이 시나리오를 바탕으로 각기 다른 계층에서 서버들의 클러스터에 요구(request)를 발송(dispatch)하는 많은 방법이 있다는 것을 알 수 있다. 일반적으로 이러한 서버들은 같은 서비스와 같은 서비스 내용을 가지고 잇다. 서비스 내용은 각 서버의 로칼 디스크(local disk)에 복제되거나 넷트웍 파일 시스템으로 공유되어지거나 분산 파일 시스템으로 제공된다. 기존의 요구 발송 기술들(request dispatching techniques)은 다음과 같은 범주들로 나눌 수 있다.

- **클라이언트 측면의 방법**

  버클리(Berkeley)의 스마트 클라이언트(Smart Client)[1]는 클라이언트 측면에서 동작하는 애플릿(applet)을 제공한다. 애플릿이 서버들의 클러스터에게 모든 서버들의 부하 정보(load information)를 모아 줄것을 요청한 후, 그 정보를 바탕으로 한 서버를 선택하여 그 서버에 요구(request)를 보낸다. 애플릿은 선택한 서버가 사용 불능 상태(down)가 되었다면 다른 서버를 시도한다. 보스턴 대학(Boston University)은 클라이언트쪽에서 대역폭(bandwidth)을 시험하여 동적으로 서버를 선택하는 기술[2]을 개발했다. 이 기술에서는 주어진 경로를 따라 가능한한 큰 대역폭과 경로상의 현재의 정체(congestion)를 실험을 통해 추정한다. 그러나 이러한 클라이언트 측면의 접근은 클라이언트-투명성을 가지지 못한다. 이들은 클라이언트 응용프로그램의 수정이 필요하기 때문에 모든 TCP/IP 서비스에 응용될 수는 없다. 게다가 이들은 부가적인 질문 또는 실험 때문에 넷트웍 정보 전송량을 증가시킬 가능성이 있다.

- **서버 측면의 Round-Robin DNS 방법**

  NCSA 확장 가능(scalable) 웹 서버는 Round-Robin DNS 방법[3, 4, 5]을 이용한 최초의 확장 가능 웹 서버의 원형이다. RRDNS 서버는 round-robin 방식으로 각기 다른 IP 주소들을 하나의 이름에 대응(map)시켜 이상적인 상황에서는 각기 다른 클라이언트들이 클러스터 안의 각기 다른 서버에 접근한다. 이런 방법으로 부하(load)가 서버들 사이에 분산된다. 그러나 클라이언트의 캐슁(caching)과 DNS 시스템의 상속적 특성 때문에 서버들 사이에 동적 부하 불균형이 초래되기 쉽다. 따라서 서버들의 최고(peak) 부하를 제어하기가 쉽지 않다. PR-DNS에서 이름 대응(name mapping)의 생존 시간(Time To Live, TTL) 값을 잘 선택하기가 어렵다. 적은 값으로는 PR-DNS가 병목을 일으킬 것이고, 높은 값으로는 동적 부하 불균형이 더욱 나빠질 것이다. TTL 값을 영으로 설정한다 하더라도 작업 할당의 기초(scheduling granuarity)는 호스트 기반이고, 어떤 사람은 사이트에서 많은 정보를 받아가고, 또 다른 사람은 몇 개의 정보만 흩어보다 가버릴 수 있기 때문에 사용자들의 각기 다른 접속 양식이 동적 부하 불균형을 초래할 것이다. 게다가 서버 노드에 장애가 있거나, 클라이언트들이 IP 주소에 대응되는 서버를 찾아가 그 서버가 사용 불능 상태(down)인것을 발견했을 때 그리 믿을만하지 못하고, 브라우저에서“reload" 나 "refresh" 버튼을 누를때조차 아직 문제가 남아있다.

● **서버 측면의 응용 프로그램 계층 작업 할당 방법**

EDDIE[6], Reverse-proxy[7], pWEB[8] 그리고 SWEB[9]는 확장 가능한(scalable) 웹 서버를 구축하기 위해 응용 프로그램 계층 작업 할당(scheduling) 방법을 이용한다. 이들은 모두 HTTP 요청을 클러스터에 있는 다른 웹 서버에 전달한다. 그 후 결과를 얻어 최종적으로 그 결과를 클라이언트에 돌려준다. 그러나 이런 방법은 각 요구(request)에 클라이언트와 부하 분산기(load balancer) 사이의 접속과 부하 분산기(load balancer)와 서버 사이의 접속의 두개 TCP 접속을 만들어야 하므로 지연(delay)이 높다. 그리고 응용 프로그램 계층에서의 HTTP 요구들과 반응들(replies)을 다루는 간접 비용 또한 높아서 서버 노드의 수가 증가하면 오래 지나지 않아 응용 프로그램 계층의 부하 분산기(load balancer)가 새로운 병목 구간이 된다.

● **서버 측면의 IP 계층 작업 할당 방법**

버클리(Berkeley)의 MagicRouter[10]와 시스코(Cisco)의 LocalDirector[11]는 다른 서버들상의 병렬 서비스들을 단일 IP 주소상의 하나의 가상 서비스로 보이도록 넷트웍 주소 번역(Network Address Translation) 방법을 사용한다. 부하 분산기(load balancer)가 요청 패킷들(request packets)의 목적지 주소(destination address)를 바꿔 선택된 서버로 보내준다. 그 후 응답 패킷들(reply packets)의 발생지 주소(source address)를 원래 요청 패킷의 목적지 주소로 바꿔준다. 그러나 MagicRouter는 다른 목적의 사용자에게는 별로 유용하지 않고 LocalDirector는 너무 비싸다. 그리고 이들은 단순히 TCP 전송 규약(protocol) 부분만을 지원할 뿐이다. 이 방법은 리눅스 가상 서버에 잘 준비되어 있고 자세한 것들은 다음 장에서 논의할 것이다.

IBM의 TCP router[12]는 IBM의 확장 가능한(scalable) 병렬 SP-2 시스템에 확장 가능한 웹 서버를 구축하기 위해 변형된 넷트웍 주소 번역 방법(modified Network Address Translation)을 사용한다. TCP router는 요청 패킷들의 목적지 주소를 바꿔 선택된 서버로 보내준다. 이 서버는 응답 패킷들의 발생지 주소를 자신의 주소 대신 TCP router 주소로 넣도록 변형되어있다. 이 변형 방법의 장점은 TCP router가 응답 패킷을 변경(rewriting)해야만 하는 것을 피할 수 있다는 것이다. 단점은 클러스터의 모든 서버들의 커널 코드(kernel code)를 변경할 필요가 있다..

ONE-IP[13]는 또 다른 IP-계층 작업 할당 방법이다. 이 방법에서는 모든 서버들이 넷트웍에서 그들 자신의 IP 주소를 가지고 IP 별칭 인터페이스(IP alias interface)에 모두 같은 IP 주소를 구성해야만 한다. 두가지 발송(dispaching) 기술이 사용된다. 한가지는 중앙의 발송기(dispacher)가 IP 패킷들을 다른 서버들로 직접 배송(routing)하는 것이고, 다른 하나는 패킷들을 흩뿌려(broadcast) 지역적으로 걸러내는(local filtering) 것이다. 응답 패킷을 변경(rewriting)하는 것을 피할 수 있다는 장점이 있다. 단점은 어떤 운영체제는 넷트웍 인터페이스가 IP 주소 충돌을 발견했을 때 정지(shutdown)하기 때문에 모든 운영체제에 적용할 수는 없다는 것이다. 지역적 걸름(local filtering) 역시 모든 서버의 커널 코드를 변경할 필요가 있다.

# 3. 가상 서버 구조

리눅스 가상 서버[14]는 현재 두가지 방식을 지원한다. 하나는 넷트웍 주소 번역(Network Address Translation)에 의한 가상 서버이고 다른 하나는 IP 터널링(IP tunneling)에 의한 가상 서버이다. 이어지는 두 절에서는 이들의 구조와 작동 원리상의 차이를 각각 설명한다. 세 번째 절에서는 이들의 장단점을 논의한다. 가상 서버 코드(code)는 현재 리눅스 커널 2.0의 리눅스 IP 매스큐래이딩(IP Masquerading) 코드에서 개발되었으며 Steven Clarke의 포트 포워딩(port forwarding) 코드를 다시 사용한다. HTTP, Proxy, DNS 등등 TCP와 UDP 양쪽 서비스들을 지원한다. 전송규약(protocol)에서 이들은 응용프로그램 데이터로 IP 주소와(또는) 포트 번호를 전달하기 때문에 이것을 제어할 추가적인 코드가 필요하다. 현재 FTP가 지원되며 다른것들은 새로운 제어 코드가 필요하다.

## 3.1 NAT에 의한 가상 서버

IPv4에서의 IP 주소 체제의 단점과 몇가지 보안상의 이유로 점점 더 많은 넷트웍들이 외부의 넷트웍(또는 인터넷)에서 사용될 수 없는 사적인 IP 주소 체제(private IP addresses)를 사용한다. 내부 내트웍의 호스트가 인터넷에 접근하려 할 때나 인터넷에서 접근되도록 하려 할 때 넷트웍 주소 번역(network address translation)의 필요성이 생긴다. 넷트웍 주소 번역은 인터넷 전송 규약의 해더들(headers)이 적절히 수정되어 클라이언트들이 각기 다른 서버들에 연결되었지만 하나의 IP 주소와 연결되고 있는 것처럼 보이고, 서버들도 클라이언트에 직접 연결된 듯 보이도록 할 수 있다는 사실에 의지한다. 이러한 성질은 가상 서버, 즉 다른 IP 주소에서의 병렬 서비스들이 NAT에 의한 하나의 IP 주소위의 하나의 가상 서비스로 보이도록 할 수 있는 서버를 구축하는데 이용될 수 있다.

NAT에 의한 가상 서버의 구조는 그림 2로 설명되어진다. 부하 분산기(load balancer)와 실제 서버들은 스위치(switch)나 허브(hub)로 상호 연결되어있다. 실제 서버들은 대개 같은 서비스를 운용하고 같은 서비스 구성 항목을 가진다. 서비스 내용은 각 서버의 지역 디스크(local disk)에 복제되거나 넷트웍 파일 시스템으로 공유되거나 (ASF나 CODA같은) 분산 파일 시스템으로 제공된다. 부하 분산기(load balancer)는 요구들(requests)을 NAT에 의해 다른 실제의 서버들로 발송(dispatch)한다.

NAT에 의한 가상 서버의 작업 흐름은 다음과 같다: 사용자가 서버 클러스터가 제공하는 서비스에 접근하면 가상 IP 주소(부하 분산기의 외부 IP 주소)로 향하는 요구 패킷(request packet)이 부하 분산기에 도달한다. 부하 분산기는 그 패킷의 목적지 주소(destination address)와 포트 번호를 검사한다. 만일 그것들이 가상 서버의 역할 분류표(virtual server rule table)에 따른 가상 서버 서비스와 일치한다면 작업 할당 방식(scheduling algorithm)에 따라 클러스터로부터 실제 서버를 선택한다. 그리고 이 접속을 모든 성립된 접속들(established connections)을 기록하는 해쉬 테이블(hash table)에 추가해 넣는다. 그 후 그 패킷의 목적지 주소와 포트 번호를 선택된 서버의 것으로 변경(rewrite)한다. 그러면 그 패킷은 선택된 서버로 전송된다. 해쉬 테이블에서 발견될 수 있는 이 접속과 성립된 접속에

속하는 패킷이 들어온다면 그 패킷은 변경(rewrite)되어 선택된 서버로 전송된다. 응답 패킷(reply packet)이 돌아오면 부하 분산기는 그 패킷의 발생지 주소(source address)와 포트를 가상 서비스의 것으로 변경(rewrite)한다. 접속이 종료하거나 시간 초과되면 접속기록은 해쉬 테이블에서 지워진다.

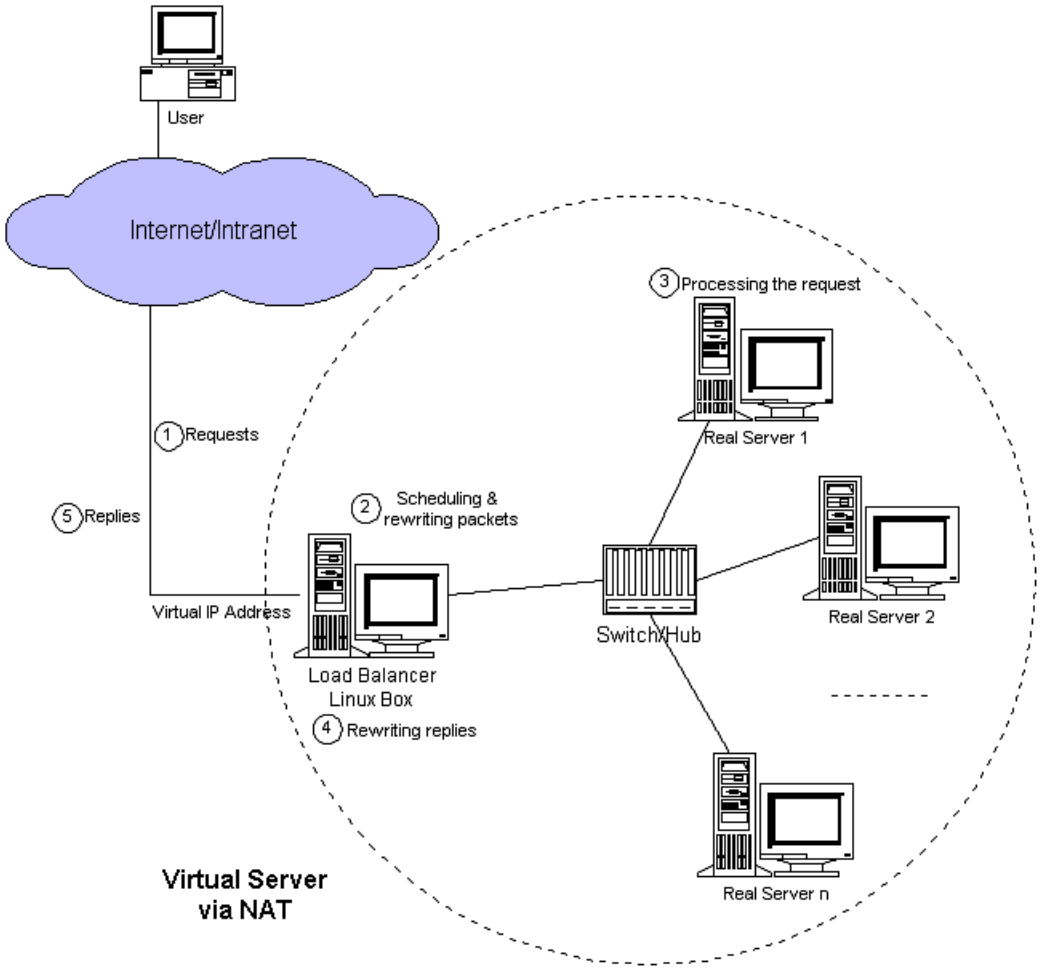



그림 1. NAT에 의한 가상 서버의 구조

NAT에 의한 가상 서버의 예가 그림 3에 있다.

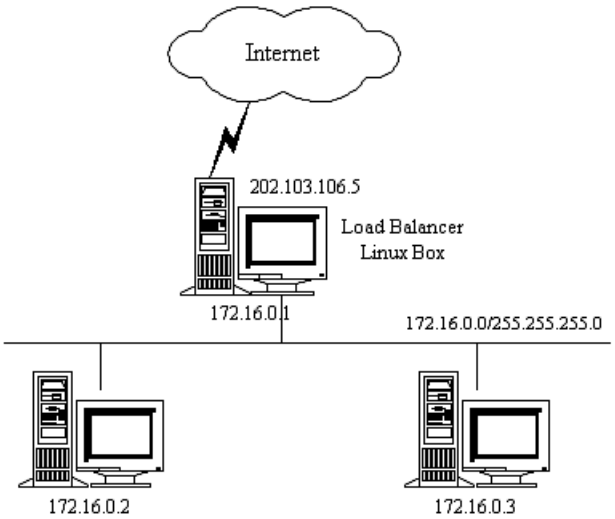



그림 1. NAT에 의한 가상 서버의 예

표 1은 가상 서버를 지원하는 리눅스 컴퓨터에서 기술된 규칙들을 보여준다.

| 표 1: 가상 서버 규칙의 예 | | | | | |
|---|---|---|---|---|---|
| 전송 규약 | 가상 IP 주소 | 포트 | 실제 IP 주소 | 포트 | 가중치 |
| TCP | 202.103.106.5 | 80 | 172.16.0.2 | 80 | 1 |
| | | | 172.16.0.3 | 8000 | 2 |
| TCP | 202.103.106.5 | 21 | 172.16.0.3 | 21 | 1 |

목적지가 IP 주소 202.103.106.5 포트 80번인 모든 정보 전송은 실제 주소 172.16.0.2 포트 80번과 172.16.0.3 포트 8000번으로 부하 분산(load-balance)된다. 목적지가 IP 주소 202.103.106.5 포트 21번인 정보 전송은 실제 IP 주소 172.16.0.3 포트 21번으로 포트 이동(port forward)된다. 실제 서버들은 TCP/IP를 지원하는 어떤한 운영체제상에서도 운용될 수 있지만 실제 서버의 default route는 반드시 가상 서버(이 예에서는 172.16.0.1이다.)여야 한다는것에 주의하라.

패킷 변경(rewriting)은 다음과 같이 동작한다.

웹 서비스에 들어오는 패킷은 다음과 같이 발생지와 목적지를 가졌을 것이다.

| 발생지 | 202.100.1.2:3456 | 목적지 | 202.103.106.5:80 |
|---|---|---|---|

부하 분산기(load balancer)가 예를 들어 172.16.0.3:8000 같은 실제 서버를 선택한다. 패킷은 다음과 같이 변경(rewrite)되어 실제 서버로 보내질 것이다.

| 발생지 | 202.100.1.2:3456 | 목적지 | 172.16.0.3:8000 |
|---|---|---|---|

다음과 같이 부하 균형 관리 컴퓨터로 응답이 되돌아 온다.

| 발생지 | 172.16.0.3:8000 | 목적지 | 202.100.1.2:3456 |
|---|---|---|---|

이 패킷은 다음과 같이 가상 서버의 주소가 기록되어 클라이언트에게 돌려준다.

| 발생지 | 202.103.165.5:80 | 목적지 | 202.100.1.2:3456 |
|---|---|---|---|

## 3.2 IP 터널링에 의한 가상 서버

IP 터널링(IP 감싸넣기; IP encapsulation; IP tunneling)은 IP화된 정보(IP datagram)안에 IP화된 정보(IP datagram)을 감싸넣는(encapsulate)는 기술이다. 이것은 목적지가 어떤 IP 주소인 정보(datagram)을 감싸서(wrape) 다른 IP 주소로 재지향(redirect) 시킬 수 있다. 이 기술을 가상 서버가 요청 패킷을 다른 서버로 짚어 넣어 주는데(tunnel) 사용할 수 있다. 그 서버는 요구들(requests)을 처리하여 그 결과를 클라이언트에게 직접 돌려준다. 그래도 서비스들은 여전히 하나의 IP 주소상의 하나의 가상 서비스로 보여지는게 가능하다.

IP 터널링에의한 가상 서버의 구조는 그림 4로 설명한다. IP 터널링에 의한 가상 서버와 NAT에 의한 가상 서버와의 가장 큰 차이점은 전자는 부하 분산기(load balancer)가 IP 터널을 통해 요구들을 실제 서버들에게 보내고 후자는 부하 분산기가 넷트웍 주소 번역(network address translation)에 의해 요구들을 실제 서버들에게 보낸다는 것이다.

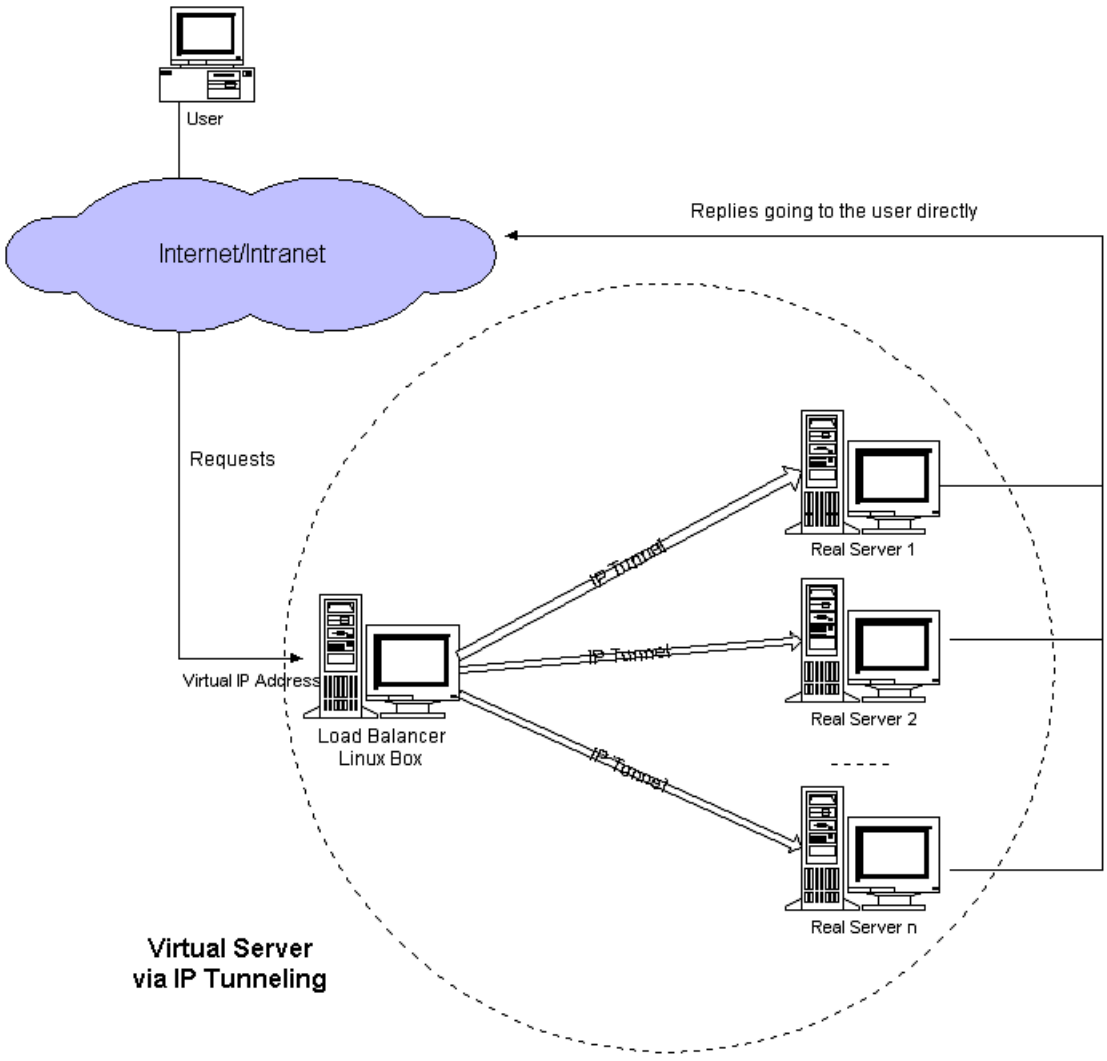



그림 1. IP 터널링에 의한 가상 서버의 구조

IP 터널링에의한 가상 서버의 동작 흐름은 다음과 같다: 사용자가 서버 클러스터가 제공하는 서비스에 접근하면 가상 IP 주소(부하 분산기의 외부 IP 주소)로 향하는 요구 패킷(request packet)이 부하 분산기에 도달한다. 부하 분산기는 그 패킷의 목적지 주소와 포트 번호를 검사한다. 만일 그것들이 가상 서버의 역할 분류표(virtual server rule table)에 따른 가상 서버 서비스와 맞는다면 작업 할당 방식에 따라 클러스터로부터 실제 서버를 선택한다. 그리고 이 접속을 모든 성립된 접속들(established connections)을 기록하는 해쉬 테이블(hash table)에 추가해 넣는다. 그런 후 부하 분산기(load balancer)가 패킷을 IP화된 정보(IP datagram)안에 감싸 넣고(encapsulate) 선택된 서버로 전송한다. 해쉬 테이블에서 발견될 수 있는 이 접속과 성립된 접속에 속하는 패킷이 들어온다면 그 패킷은 다시 감싸 안겨져(encapsulate) 선택된 서버로 전송된다. 서버가 감싸 안겨진 패킷을 받으면 그 패킷을 풀어 해쳐(decapsulate) 요청을 처리하고, 마지막으로 그 결과를 사용자에게 직업 돌려준다. 접속이 종료하거나 시간 초과되면 접속기록은 해쉬 테이블에서 지워진다.

실제 서버들은 어떠한 넷트웍에서 어떠한 실제의 IP 주소를 가질수도 있다. 이들은 지리적으로 분산되어 있을 수도 있다. 그러나 실제 서버들은 반드시 IP 감싸넣기 전송 규약(IP encapsulation protocol)을 지원해야하고 리눅스에서는 “ifconfig tunl0 <가상 IP 주소>” 로 터널 장치(tunnel device)를 구성하는 것처럼 그들 모두는 <가상 IP 주소>로 구성된 터널 장치중 하나를 가져야한다는 것을 주의하라. 감싸 안겨진 패킷(encapsulated packet)이 도달하면 실제의 서버는 그것을 풀어 헤쳐(decapsulate) 그 패킷이 <가상 IP 주소>가 목적지임을 발견한다. 그래서 요청을 처리하고 최종적으로 그 결과를 사용자에게 직접 돌려준다.

## 3.3 장점과 단점

### 3.3.1 NAT에 의한 가상 서버

NAT에 의한 가상 서버의 장점은 실제 서버가 TCP/IP를 지원하는 어떠한 운영체제에서도 운용될 수 있다는 것이다. 실제 서버는 사적 인터넷 주소체제(private Internet addresses)를 사용할 수 있고 단지 부하 분산기(load balancer)만 하나의 IP 주소가 필요하다.

단점은 NAT에 의한 가상 서버의 가용성이 아주 좋은 것은 아니라는 점이다. 서버 노드의 수가 약 25개나 그 이상 늘어날 때 실제 서버들의 처리 능력(throughout)에 의존하는 부하 분산기가 전체 시스템의 병목 구간이 될 수 있다. 요청 패킷들(request packets)과 응답 패킷들(reply packets) 양쪽을 부하 분산기(load balancer)가 변경(rewrite)할 필요가 있기 때문이다. 만약 TCP 패킷의 평균 길이가 536 Bytes 라면 패킷의 변경에의한 평균 지연 시간은 (Pentium 프로세서에서, 더 빠른 프로세서를 사용한다면 조금은 줄어들 수 있지만) 약 60us 정도고 부하 분산기(load balancer)의 최대 처리 능력(throughout)은 8.93 Mbytes/s 일 것이다. 만일 실제 서버들의 평균 처리 능력이 400KBytes/s 라면 부하 분산기(load balancer)가 22개의 실제 서버들을 작업 할당(schedule)할 수 있다.

NAT에 의한 가상 서버가 많은 서버들을 관리해야할 수도 있다. 만일 부하 분산기가 전체 시스템의 병목 구간이 된다면 이를 해결하는 두가지 방법이 있다. 한가지는 IP 터널링(IP tunneling)에 의한 가상 서버이고 다른 하나는 혼합적(hybrid) 방법이다. 혼합적 방법에서 모두 그들 자신의 서버 클러스터를 가지는 다수의 부하 분산기들을 도입하고 그 부하 분산기들을 Round-Round DNS에 의해 하나의 도메인 이름(domain name)으로 묶는다.

.

### 3.3.2 IP 터널링에 의한 가상 서버

입력 패킷들(incoming packets)은 언제나 작고 응답 패킷들(reply packets)은 언제나 많은 양의 데이터를 가져가는 (웹 서비스같은) 많은 인터넷 서비스들이 있다. IP 터널링에 의한 가상 서버에서는 부하 분산기(load balancer)가 단지 다른 실제 서버들에 요청들을 작업 할당(schedule)할 뿐이다. 그리고 실제 서버들은 응답들을 사용자에게 직접 돌려준다. 따라서 부하 분산기는 많은 양의 요구들(requests)을 다룰 수 있다: 부하 분산기가 100대가 넘는 실제 서버들에 작업을 배정하여도 시스템의 병목 구간이 되지 않는다. 그러므로 IP 터널링(IP tunneling)을 사용하면 부하 분산기에 최대 서버 노드 수를 대단히 많이 증가시킬 수 있다. 부하 분산기가 단지 100Mbps의 전복조(full-duplex) 넷트웍 어댑터(network adapter)를 가졌다 하더라도 가상 서버의 최대 처리 능력은 1Gbps에 달할 수 있다.

IP 터널링의 성질은 대단히 고성능의 가상 서버를 구척하는데 이용될 수 있고, 가상 프락시 캐쉬 서버(virtual proxy cache server)를 구축하는데 대단히 좋다. 프락시 서버가 요구를 받으면 대상들을 가져오기 위해(to fetch objects) 인터넷에 직접 접근할 수 있고 사용자들에게 그것을 직접 돌려줄 수 있기 때문이다.

그러나 IP 터널링에 의한 가상 서버에서는 실제 서버들이 IP 터널링 전송 규약(IP tunneling protocol)을 지원할 필요가 있다. 이 방법의 특성은 리눅스가 운용되는 서버들에서 시험되었다. IP 터널링 전송 규약이 모든 운영 체제의 표준이 되어가고 있기 때문에 IP 터널링에 의한 가상 서버가 다른 운영 체제에서 운용되는 서버들에서 응용될 수 있게 될 것이다.

# 4. 작업 할당 방식(Scheduling Algorithms)

클러스터로부터 서버를 선택하기위한 네 개의 작업 할당 방식들(scheduling algorithms)이 준비되어있다: Round-Robin, 가중치가 있는(weighted) Round-Robin, 최소-접속(Least-Connection) 그리고 가중치가 있는 최소-접속의 방식들이다. 처음의 두 방식들은 서버에대한 어떠한 부하 정보(load information)도 가지고 있지 않기 때문에 자명(self-explanatory)하다. 뒤의 두 방식들은 각 서버에대한 활동 접속 수(active connection number)를 세어 이러한 접속 수에 의해 그들의 부하를 추정한다.

## 4.1 Round-Robin 작업 할당

Round-robin 작업 할당 방식(scheduling algorithm)은 그 용어의 의미대로 넷트웍 접속을 round-robin 방식으로 각기 다른 서버들에 보낸다. 이 방식은 실제의 서버들을 접속의 수나 응답 시간을 고려하지 않고 모두 동일한 것으로 취급한다. Round-robin DNS가 이런식으로 동작한다해도 상당히 다른 면이 있다. Round-robin DNS는 하나의 도메인(domain)을 각기 다른 IP 주소들로 해석해준다. 작업 할당 기초(scheduling granularity)가 호스트 기반이라 DNS의 캐슁은 알고리즘의 효과를 방해한다. 이는 실제 서버들 사이에서 충분히 동적 부하 불균형을 있을 수 있게한다. 가상 서버의 작업 할당 기초는 넷트웍 접속 기반이고, 좋은 작업 할당 기초덕에 round-robin DNS보다 우수하다.

## 4.2 가중치가 있는(weighted) Round-Robin 작업 할당

가중치가 있는 round-robin 작업 할당(scheduling)은 각기 다른 처리 용량의 실제 서버들을 다룰 수 있다. 각 서버들에 처리 용량을 나타내는 정수로된 가중치가 할당(assign)되어있다. 기본 설정 가중치는 1이다. 예를 들어 실제 서버들 A, B, 그리고 C가 각각 가중치 4, 3, 2를 가진다면 좋은 작업 할당 순서(scheduling sequence)는 한 작업 할당 주기(scheduling period) (mod sum(Wi))에서 ABCABCABA이다. 가중치가 있는 round-robin 작업 할당을 도입하면 가상 서버의 규칙들이 변경된 후에 서버 가중치에 따라 작업 할당  순서가 생성된다. 넷트웍 접속은 작업 할당 순서에 따라 Round-robin 방식으로 각기 다른 실제 서버들로 보낸다.

가중치가 있는 round-robin 작업 할당은 각 실제 서버들에 대한 넷트웍 접속을 세어(count) 줄 필요가 없고 작업 할당의 간접 비용이 동적 작업 할당 방식보다 적어 보다 많은 실제 서버들을 가질 수 있다. 그러나 만일 요청의 부하가 대단히 높다면 실제 서버들 사이에 동적 부하 불균형을 초래할 수 있다. 간단히 말해 대부분의 긴 요구들(long requests)이 하나의 실제 서버로 보내지는 일 또한 있을 수 있다.

## 4.3 최소-접속 작업 할당

최소-접속 작업 할당 방식(least-connection scheduling algorithm)은 넷트웍 접속들을 가장 적은 수의 활동 접속(active connection)을 가진 서버로 보낸다. 이것은 각각의 활동 접속을 동적으로 세어야(count)할 필요가 있기 때문에 동적 작업 할당 방식의 하나이다. 비슷한 성능의 서버들을 모아둔 가상 서버에서는 모든 긴 요구들(long requests)이 하나의 실제 서버로 보내지는 일이 없기 때문에 최소-접속 작업 할당은 요구들(requests)의 부하(load)가 대단히 많을 때도 배분(distribution)이 부드럽다.

한눈에 최소-접속 작업 할당은 서버들의 처리 용량이 다양하더라도 빠른 서버가 더 많은 넷트웍 접속을 받아 역시 좋은 성능일것으로 생각된다. 사실은 TCP의 TIME_WAIT 상태 때문에 아주 좋은 성능일 수는 없다. TCP의 TIME_WAIT는 보통 2분이다. 이 2분동안 붐비는 웹 사이트는 종종 수천개의 접속을 받기도 한다. 예를 들어 A 서버가 B 서버보다 두 배 더 강력하다면 A 서버는 수천의 접속을 처리해 TCP의 TIME_WAIT 상태에 둔다. 그러나 B 서버는 이 수천개의 접속이 끝나기까지 지연된다. 그래서 최소-접속 작업 할당은 다양한 처리 용량의 서버들 사이에서는 부하의 균형을 잘 맞추지 못한다.

## 4.4 가중치가 있는 최소-접속 작업 할당

가중치가 있는 최소-접속 작업 할당(weighted least-connection scheduling)은 최소-접속 작업 할당의 모집합(superset)이다. 이 방법으로 각각의 실제 서버에 성능 가중치를 줄 수 있다. 높은 가중치의 서버는 어떠한 한 순간에라도 많은 비율의 활동 접속(active connection)을 받는다. 가상 서버 관리자는 각각의 실제 서버에 가중치를 부여할수 있다. 그리고 넷트웍 접속을 각 서버에 작업 할당(schedule)할 수 있다. 각 서버의 현재 활동 접속 수의 비율은 그 가중치의 비다. 기본 설정 가중치는 1이다.

가중치가 있는 최소-접속 작업 할당은 다음과 같이 동작한다:

n개의 실제 서버가 있고 각 서버 i는 가중치 $W_i$ (i = 1, ... , n)를 가진다고 가정하자. 서버 i의 활동 접속(active connection)은 $C_i$ (i = 1, ... , n)이고 *모든_접속* 은 $C_i$ (i = 1, ... , n)의 합이다. 서버 j로 가는 넷트웍 접속은 아래와 같다.

$$(C_j \ / \ 모든_접속) \ / \ W_j = \min \{(C_i \ / \ 모든_접속) \ / \ W_i\} \ (i = 1, \ ... \ , n)$$

이 비교에서 *모든_접속* 은 상수이므로 $C_i$를 *모든_접속* 으로 나눠줄 필요가 없다. 식은 다음과 같이 최적화될 수 있다.

$$C_j \ / \ W_j = \min \{C_i \ / \ W_i\} \ (i = 1, \ ... \ , n)$$

가중치가 있는 최소-접속 작업 할당 방식에서는 최소-접속에 비해 부가적인 배분(division)이 필요하다. 작업 할당의 간접 비용을 최소화하기위해 서버들이 같은 처리 용량을 가졌을 때 최소-접속 작업 할당과 가중치가 있는 최소-접속 작업 할당 양쪽 다 도구로 사용된다.

# 5. 고 가용성(High Availability)

중요한 상업적 응용 분야가 인터넷으로 점점 더 많이 옮아옴에 따라 가용성이 높은 서버들을 제공하는 것이 점점 더 중요해지고 있다. 클러스터화된 시스템의 장점중 하나는 하드웨어와 소프트웨어의 여유분이 있다는 것이다. 높은 가용성은 노드(node)나 데몬(deamon)의 장애들(failures)이나 시스템의 재구성(reconfiguring)을 적절히 감지해 클러스터에 남아있는 노드로 작업 부하(workload)를 이전하는 것으로 이룰 수 있다. 가상 서버의 높은 가용성을 현재 “mon"과 ”fake" 소프트웨어를 사용해 제공한다. “mon"은 넷트웍 서비스 가용성과 서버 노드를 모니터할 수 있는 범용 자원 모니터링 시스템(general-purpose resource monitoring system)이다. "fake"는 ARP 속이기(spoofing)를 사용해 IP를 이전하는 소프트웨어다.

서버의 장애 극복(failover)은 다음과 같이 제어한다: 부하 분산기(load balancer)에 클러스터의 서비스 데몬들과 서버 노드들을 모니터하기 위한 "mon" 데몬이 운영된다. 매 t 초마다 서버 노드들이 살아있는지 감지하도록 fping.monitor를 구성하고 매 m 분마다 모든 노드의 서비스 데몬들을 감지하기 위해 연관된 모니터 프로그램 역시 구성한다. 예를 들어 http 서비스를 점검하기위해 http.monitor를, ftp 서비스를 위해 ftp.monitor를 또, 기타 다른 모니터 프로그램을 사용할 수 있다. 서버 노드나 데몬이 새로 사용 불능 상태(down)가 되거나 사용 가능 상태(up)가 되면 가상 서버 테이블에 규칙을 제거하거나 추가하기위한 경보가 작성된다. 따라서 부하 분산기가 서비스 데몬이나 서버를 제거하거나 복구되었을 때 서비스에 추가하는 것을 자동으로 할 수 있다.

이제 부하 분산기는 장애가 있으면 전체 시스템에 영향을 미치는 곳(single failure point)이 되었다. 부하 분산기의 장애를 방지하기 위해 부하 분산기의 백업(backup) 서버를 설치한다. 부하 분산기에 장애가 있을 때 부하 분산기의 IP 주소를 백업 서버로 이전하기위해 “fake" 소프트웨어가 사용된다. 또, 백업 서버에서는 부하 분산기의 상태를 검사하여 ”fake"를 실행하거나 실행 중단하기위해 "mon"을 사용한다. “mon" 데몬은 백업 서버에서도 또한 운용되어 백업 서버가 클러스터의 현재 상태를 알고 있어야 한다. 주 부하 분산기에 장애가 있을 때 백업 서버가 그 IP 주소(들)를 인수하여 서비스들을 계속 제공한다. 그러나 해쉬 테이블의 성립된 접속(established connection)은 소멸되며 클라이언트들은 그들의 요구들(requests)을 다시 보내야할 필요가 있다.

# 6. 결론과 미래에 해야할 일

우리는 어떻게 실제 서버들의 클러스터에 기초한 리눅스 가상 서버를 만드는지를 제시하였다. 클러스터의 병렬 서비스들을 하나의 IP 주소상의 하나의 가상 서비스로 보이게하기 위한 IP 계층 부하 분산(load balancing)의 두가지 방법이 개발되었다. 하나는 넷트웍 주소 번역(Network Address Translation)에 의한 가상서버이고 다른 하나는 IP 터널링(IP tunneling)에 의한 가상서버이다. 현재 각기 다른 응용 상황들에 맞춰 네가지 작업 할당 방식(scheduling algorithm)이 개발되어 있다. 확장성(scalability)은 클러스터에 노드(node)를 투명하게 추가하거나 제거하는 것으로 이루어진다. 고 가용성(high availability)은 노드나 데몬(daemon)의 장애(failure)와 시스템의 재구성(reconfiguring)을 적절히 검사함으로써 제공된다. 이 솔루션(solution)들은 클라이언트나 서버에 아무런 변경도 요구하지 않고 거의 모든 TCP와 UDP 서비스들을 지원한다.

미래에는 부하-정보를 이용한 작업 할당(load-informed scheduling), IP 터널링에 의한 가상 서버에서 위치에 의한 작업 할당(geographic-based scheduling)같은 다른 더 많은 필요들에 맞는 더 많은 부하 분산 방식을 연구하고 추가하게 될 것이다. “heartbeat" 코드와 CODA 분산 고장 감내 파일시스템(distributed fault-tolerant filesystem)을 가상 서버로 통합하고 클러스터 관리 프로그램을 개발하여 리눅스 가상 서버의 설치와 관리가 쉽도록 만들려한다. 높은 수준의 고장 감내를 연구하게 될 것이다: 트랜젝션(transaction)과 로깅(logging) 과정[17]을 부하 분산기(load balancer)에 추가해, 부하 분산기가 다른 서버에 요청을 다시 보낼 수 있어 클라이언트가 요청을 다시 보낼 필요가 없도록 하고 주 부하 분산기와 백업 부하 분산기가 그들의 상태를 교환하여 백업 서버가 인계받을 때 현존하는 접속이 소실되지 않도록 노력하게 될 것이다. 어떻게 IPv6에서 가상 서버를 실행할것인가에 대해서도 역시 연구하게 될 것이다.

## 참조


1. Chad Yoshikawa, Brent Chun, Paul Eastharn, Armin Vahdat, Thomas Anderson, and David Culler, "Using Smart Clients to Build Scalable Services," USENIX'97 (1997). http://now.cs.berkeley.edu/.
2. Robert L. Carter and Mark E. Crovella, "Dynamic Server Selection using Bandwidth Probing in Wide-Area Networks," Boston University Technical Report (1996). http://www.ncstrl.org/.
3. Eric Dean Katz, Michelle Butler, and Robert McGrath, "A Scalable HTTP Server: The NCSA Prototype," Computer Networks and ISDN Systems, pp. 155-163 (May 1994).
4. Thomas T. Kwan, Robert E. McGrath, and Daniel A. Reed, "NCSA's World Wide Web Server: Design and Performance," IEEE Computer, pp. 68-74 (November 1995).
5. T. Brisco, DNS Support for Load Balancing, RFC 1794. http://www.internic.net/ds/.
6. A. Dahlin, M. Froberg, J. Walerud, and P. Winroth, EDDIE: A Robust and Scalable Internet Server (May 1998). http://www.eddieware.org/.
7. Ralf S.Engelschall, "Load Balancing Your Web Site: Practical Approaches for Distributing HTTP Traffic," Web Techniques Magazine, 3, 5 (May 1998). http://www.webtechniques.com/.
8. Edward Walker, pWEB - A Parallel Web Server Harness (April, 1997). http://www.ihpc.nus.edu.sg/STAFF/edward/pweb.html.
9. Daniel Andresen, Tao Yang, and Oscar H. Ibarra, "Tow ards a Scalable Distributed WWW Server on Workstation Clusters," Proc. of 10th IEEE Intl. Symp. Of Parallel Processing (IPPS'96), pp. 850-856 (Arpil 1996). http://www.cs.ucsb.edu/Research/rapid_sweb/SWEB.html.
10. Eric Anderson, Dave Patterson, and Eric Brewer, The Magicrouter: an Application of Fast Packet Interposing (May 1996). http://www.cs.berkeley.edu/~eanders/magicrouter/.
11. Cisco System, Cisco Local Director (1998). http://www.cisco.com/warp/public/751/lodir/index.html.
12. D. Dias, W. Kish, R. Mukherjee, and R. Tew ari, "A Scalable and Highly Available Server," COMPCON 1996, pp. 85-92 (1996).
13. Om P. Damani, P. Emerald Chung, and Yennun Huang, ONE-IP: Techniques for Hosting a Service on a Cluster of Machines (August 1997). http://www.cs.utexas.edu/users/damani/.
14. Wensong Zhang, Linux Virtual Server Project (May 1998). http://proxy.iinchina.net/~wensong/ippfvs/.
15. Jim Trocki, mon: Service Monitoring Daemon (July 1998). http://consult.ml.org/~trockij/mon/.
16. Simon Horman, "Creating Redundant Linux Servers," The 4th Annual LinuxExpo Conference (May 1998). http://linux.zipworld.com.au/fake/.
17. Jim Gray and T. Reuter, Tr ansaction Processing Concepts and Techniques, Morgan Kaufmann (1994).